Sevjala — Project Blueprint v1

Oct 12, 2026 · @jufi

Overview

Sevjala is a LAN-first database backup and replication manager for MySQL/MariaDB on Windows (especially XAMPP), shipped as one Go binary, operated from a CLI and tray, with a browser dashboard in Phase 2. No cloud, no Internet, no database-admin knowledge required.

Naming

Name	Meaning
Sevjala	Product name (Sev from Sevtee, jala = net/network)
Sevjala Agent	The service process, `sevjala agent`; same code on every machine
Sevjala Controller	Agent role on the database server: schedules, dumps, distributes, monitors
Sevjala Node	Agent role on backup machines: receives, verifies, stores, reports
Sevjala CLI	`sevjala` command-line tool (same binary)
Sevjala UI	Browser dashboard served on 127.0.0.1 by `sevjala ui` (Phase 2)

Descriptor: **Sevjala — Database Backup & Replication**.

Targets: Windows 7 (with KB3125574), Windows 10, Windows 11, macOS, Linux. Database: MySQL/MariaDB, XAMPP detection first. 32-bit and 64-bit Windows builds.

Priorities, in order: data safety, reliability, simple recovery, transparent status, minimal operator intervention.

Design principle kept from the PRD: replication, backup creation, backup distribution, and monitoring are separate capabilities that interact but are never one undifferentiated feature. Sevjala never implements database replication itself.

Architecture decisions

These are locked. An implementing agent follows them without re-litigating; a change needs a new row and a reason.

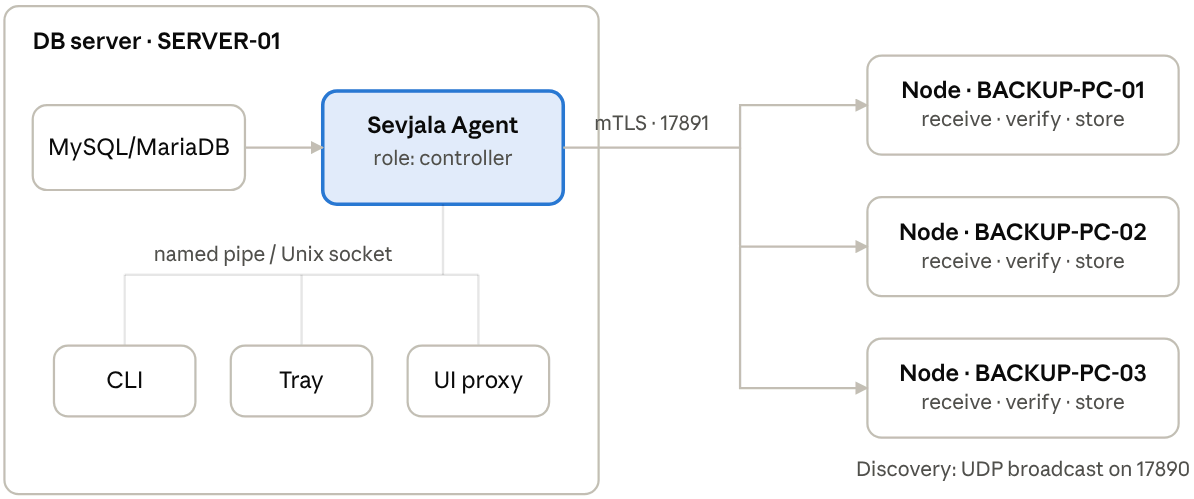
ID	Decision	Why
AD-01	One binary, one agent codebase; role (`controller` / `node`) is config, not a build	Shared discovery, identity, pairing, TLS; trivial cross-platform; a node can be promoted later
AD-02	The Controller is the only initiator on the LAN; Nodes are passive HTTPS servers	Collapses heartbeat, inventory report, ACK, and push/pull into one call pattern; Nodes never need the Controller's address
AD-03	Push model: Controller `PUT`s backups to Nodes	Controller owns the schedule and the file; Nodes stay dumb stores
AD-04	Single LAN protocol: HTTP/1.1 over mutual TLS on one TCP port; UDP broadcast only for discovery	Go stdlib only; one port to open in firewalls; no custom framing
AD-05	Discovery = UDP subnet broadcast + manual add fallback; no mDNS/multicast	Windows 7 has no mDNS; multicast is unreliable on cheap switches
AD-06	Pairing = self-signed cert per agent, pinned fingerprints, bootstrapped by a 6-digit code	No CA, no token rotation; TLS from day one at zero cost
AD-07	State: Controller store is authoritative for *expected*; each Node's directory listing is the truth for *actual*; reconciliation = diff	Nodes need no database; reinstall a Node and it recovers from its folder
AD-08	Retention computed centrally, enforced remotely via `DELETE`; Nodes delete only on instruction	One policy engine, no two-sided reconciliation
AD-09	One dump per tick, tagged with every policy it satisfies; retention promotes	Avoids 4 dumps at 01:00; cuts load on the production DB
AD-10	Controller keeps a local staging copy of recent backups	Recovery point zero; source for offline backfill and future self-heal

ID	Decision	Why
AD-11	Single-pass streaming: `mysqldump → gzip → (file, sha256)`; Node hashes while writing; temp file + atomic rename	No file read twice anywhere; a killed transfer leaves no partial file
AD-12	No resumable chunks in Phase 1; whole-file retry	LAN + files of a few hundred MB; simplicity wins
AD-13	Local control channel = Windows named pipe / Unix socket, HTTP over it; CLI, tray and UI proxy use it	No extra port on Windows; OS ACL is the auth
AD-14	UI = browser dashboard on `127.0.0.1` served by an on-demand `sevjala ui` process that reverse-proxies to the pipe; Svelte 5 + Vite, embedded in the binary	No native toolkit; nothing listens when not in use; not reachable from the LAN
AD-15	Toolchain: stock Go for win10+/macOS/Linux; XTLS/go-win7 patched SDK for Windows 7 builds (amd64 + 386), pinned by checksum	Modern Go on Windows 7; one source tree
AD-16	Service management via `kardianos/service`; tray via `fyne.io/systray`	Windows Service / launchd / systemd behind one interface; pure-Go tray on Windows
AD-17	Replication monitoring, PITR, self-heal, notifications are Phase 3	Keep the MVP to discover, pair, back up, verify, retain, restore

Components

One binary, `sevjala`, provides every process; which code runs is decided by the subcommand and the configured role.

The Controller drives everything; Nodes and local tools only answer



Sevjala components · one agent, two roles, three interfaces

The Controller agent dumps the local database, pushes copies to Nodes over mTLS, and answers local tools over the pipe. Nodes and local tools never initiate anything on the LAN.

Process	Command	Runs as	Talks to
Agent (controller role)	`sevjala agent`	Service, always on	MySQL (mysqldump/mysql), Nodes over mTLS, local pipe server
Agent (node role)	`sevjala agent`	Service, always on	Nothing outbound; serves mTLS API and local pipe
CLI	`sevjala <cmd>`	On demand, user	Local pipe only
Tray	`sevjala tray`	At user login	Local pipe; polls status, launches `sevjala ui`
UI proxy	`sevjala ui`	On demand, user	Listens on 127.0.0.1, proxies `/api` to the pipe, serves embedded Svelte build

Internal packages (Go): `identity` (agent ID, cert), `discovery` (UDP), `pairing`, `lanapi` (mTLS server + client), `localapi` (pipe server), `dump` (mysql detection, dump pipeline), `transfer`, `store` (controller state), `inventory` (node directory + manifests),

`scheduler`, `retention`, `reconcile`, `restore`, `svc` (service wrapper), `tray`, `ui`.

Coordination protocol

Every agent, whatever its role, boots the same way: load or create identity, listen for discovery on UDP 17890, serve HTTPS on TCP 17891, serve the local pipe. Only a controller-role agent ever opens outbound connections.

Identity

- `agent_id`: UUIDv4, generated once, persisted in the data dir. The Controller keys everything by `agent_id`, never by IP.
- Self-signed X.509 cert + key (ECDSA P-256), generated once, 10-year validity, CN = `agent_id`. Fingerprint = SHA-256 of the DER cert, shown as `sha256:<hex>`.

Discovery (Controller → all)

1. Controller sends to the subnet broadcast address of every non-loopback IPv4 interface: `SEVJALA/1 DISCOVER <nonce>`.
2. Each agent replies by UDP unicast to the sender: `{"v":1,"nonce":"…","agent_id":"…","hostname":"…","port":17891,"role":"node","version":"0.1.0","os":"windows","fp":"sha256:…"}`.
3. Controller collects for 2 s, dedupes by `agent_id`, discards replies with a wrong nonce.
4. Fallback: `sevjala agents add <host:port>` fetches the same metadata over TLS (`GET /identity`, no client cert required).

Discovery is read-only metadata. It never authorises anything.

Pairing (code-based trust bootstrap)

1. On the node machine: `sevjala code` prints a 6-digit code valid for 10 minutes (stored in the node agent via the pipe).
2. On the controller: `sevjala pair <agent_id> --code 123456`.
3. Controller connects over TLS with `InsecureSkipVerify`, then checks the presented cert's fingerprint equals the `fp` seen in discovery; mismatch aborts.
4. Controller sends `POST /pair` with `{controller_id, controller_fp, proof}` where `proof = HMAC-SHA256(key=code, msg=controller_fp + ":" + node_fp)`.
5. Node recomputes the HMAC against its active code. Match: store `controller_fp` as its single trusted peer, invalidate the code, reply 200. Mismatch: 403, count one attempt (5 attempts burn the code).

6. Controller stores `{agent_id, hostname, last_ip, port, fp, paired_at}` in its store.

From then on every call is mutual TLS: both sides present their cert, both verify the peer's fingerprint against the pinned value, and anything else is rejected at the TLS layer. A node accepts exactly one controller in v1.

LAN API (served by every agent, used only by the Controller)

Method	Path	Auth	Purpose
GET	`/identity`	none	agent_id, hostname, role, version, fp
POST	`/pair`	TLS + HMAC proof	Establish trust (see above)
GET	`/status`	mTLS	Health, free disk, version, full backup inventory; this is the heartbeat
PUT	`/backups/{id}`	mTLS	Stream a backup; headers `X-Sevjala-Sha256`, `X-Sevjala-Size`, manifest JSON in `X-Sevjala-Manifest` (base64)
GET	`/backups/{id}`	mTLS	Stream a stored backup back (restore, backfill)
DELETE	`/backups/{id}`	mTLS	Retention; body carries `{"reason":"…"}` and is logged on the node

Controller polls `/status` every 15 s per node. Three missed polls = `offline`. A `/status` reply with `free_bytes` under the configured floor = `warning`.

Local API (pipe / socket, HTTP, no TLS)

Windows: `\\.\pipe\sevjala`. macOS/Linux: `/var/run/sevjala.sock` (0660, group `sevjala`). Same JSON style as the LAN API, plus controller-only routes: `/agents`, `/discover`, `/pair`, `/policies`, `/backups`, `/backup/now`, `/restore`, `/events` (SSE), `/logs`. The CLI, tray and UI proxy are the only clients.

Ports (all configurable): 17890/udp discovery, 17891/tcp LAN API. `sevjala service install` adds the firewall rules on Windows; `sevjala doctor` verifies them.

Backup engine

A backup is one gzip'd mysqldump, hashed once, pushed in parallel, and counted as successful per node only after that node has verified the hash and renamed the file into place.

Database detection (`sevjala db detect`)

- Look for XAMPP first: Windows `C:\xampp\mysql\bin`, macOS `/Applications/XAMPP/xamppfiles/bin`, Linux `/opt/lampp/bin`; then `mysqldump` / `mariadb-dump` on `PATH`; then a configured path.
- Connect with the configured credentials; record server flavour and version (`SELECT VERSION()`), and the list of databases.
- Per selected database, list tables with engine. If any table is not InnoDB, the dump uses `--lock-tables`; otherwise `--single-transaction`. Report this in `doctor` and `status`.

Dump pipeline (Controller)

1. Scheduler fires for a database at time T. If a dump for T is already in progress, skip.
2. Allocate `backup_id = BK-<yyyymmdd>-<hhmmss>-<4 hex>`; create `staging/<backup_id>.part`.
3. Run `mysqldump --routines --triggers --events --hex-blob --no-create-db <consistency flag> <db>`; pipe stdout through `gzip.Writer` into `io.MultiWriter(file, sha256)`. Non-zero exit or stderr containing `error` aborts and deletes the `.part`.
4. Close, rename to `staging/<backup_id>.sql.gz`, write `staging/<backup_id>.json` (manifest), insert the backup row with `status=created` and the set of policies it satisfies.
5. Hand off to the transfer worker pool.

Transfer (Controller → Node)

- One worker per target node; targets = the union of destinations of every policy the backup satisfies.
- `PUT /backups/{id}` with the file body streamed and the manifest in headers.
- Node side: write to `<store>/<backup_id>.part` while hashing; on EOF compare hash and size; match → `rename` to `<backup_id>.sql.gz`, write sidecar `<backup_id>.json`, return 200 with the hash; mismatch → delete `.part`, return 422.

- Controller records per-destination state: `pending → transferring → verified | failed | offline`. Failures retry with backoff 30 s, 2 min, 10 min, then once per `/status` cycle while the node is online.
- Any `.part` older than 1 h on a node is deleted at `/status` time and reported.

Policies and tiers (configurable; defaults below)

Policy	Schedule	Keep	Default destinations
frequent	every 10 min	12	nearest node
hourly	at :00	24	all nodes
daily	01:00	30	all nodes
monthly	1st, 01:00	12	all nodes

A single dump at 01:00 on the 1st is tagged `frequent, hourly, daily, monthly`. Retention runs per tag: a backup is deletable on a node only when no tag it carries still needs it there.

Retention and reconciliation (Controller, every 5 min)

1. For each node, compute `expected` = backups whose tags still require that node.
2. From the latest `/status`, read `actual`.
3. `actual − expected` and not held by any tag → `DELETE /backups/{id}` with reason; nodes delete only on this instruction.
4. `expected − actual` → warning in the matrix; if the Controller still has the file in staging and the node is online, re-push (offline backfill).
5. Staging on the Controller keeps the newest N (default 12) plus anything still pending on any node.

`sevjala retention run --dry-run` prints steps 3 and 4 without executing.

Restore (`sevjala restore <backup_id> --into <db>`)

1. Prefer the Controller's staging copy; otherwise `GET /backups/{id}` from the first node reporting `verified`.
2. Re-hash the file; abort on mismatch and mark that copy `corrupt`.
3. `CREATE DATABASE <db>`; `gunzip | mysql <db>`.

4. `--replace-production` instead: take a fresh dump of the current database into staging as `PRE-RESTORE-<ts>`, require the user to type the database name to confirm, then drop and reload.
5. Log start, source copy, target, duration, result.

Manual backup: `sevjala backup now [--db X] [--tag daily]` runs the same pipeline outside the schedule.

Data model and on-disk formats

The Controller owns a small SQLite database for *expected* state; every Node's truth is its directory of backup files and sidecar manifests. Nothing else is persisted.

Data directory (created by `sevjala service install`)

OS	Path
Windows	`%ProgramData%\Sevjala\`
macOS	`/Library/Application Support/Sevjala/`
Linux	`/var/lib/sevjala/`

Contents: `config.toml`, `identity/` (agent_id, cert.pem, key.pem, peers.json), `logs/` (rotating, 10 MB × 10), and per role `controller.db` + `staging/` or `store/`.

config.toml (minimal example)

```
role = "controller"          # or "node"
listen = "0.0.0.0:17891"
discovery_port = 17890

[controller.database]
host = "127.0.0.1"
port = 3306
user = "root"
password_ref = "keyring:db"  # never plaintext here
dumper = "C:\\xampp\\mysql\\bin\\mysqldump.exe"
databases = ["production_db"]

[controller.staging]
keep = 12

[node.store]
path = "D:\\SevjalaBackups"
min_free_bytes = 2147483648
```

Controller store (SQLite, `modernc.org/sqlite`, no cgo)

Table	Columns
agents	agent_id PK, hostname, last_ip, port, fp, role, version, paired_at, last_seen, state (`online/warning/offline/error`), free_bytes
policies	name PK, cron, keep, destinations (JSON list of agent_id), enabled
backups	backup_id PK, database, created_at, size, sha256, tags (JSON list), staging_path, state (`created/distributed/partial/expired`)
backup_copies	backup_id, agent_id, state (`pending/transferring/verified/failed/offline/corrupt/deleted`), attempts, last_error, verified_at; PK (backup_id, agent_id)
events	id, at, kind, agent_id, backup_id, message (feeds `/events` and the log screen)

Node store directory

```
D:\SevjalaBackups\
  BK-20261012-163000-A8F2.sql.gz
  BK-20261012-163000-A8F2.json
  BK-20261012-170000-1C9D.sql.gz
  BK-20261012-170000-1C9D.json
  .part files only while a transfer is in flight
```

`GET /status` is built by listing this directory; a sidecar without its file, or a file whose size differs from its sidecar, is reported as `corrupt`.

Manifest `<backup_id>.json` (identical on Controller and Nodes)

```
{
  "v": 1,
  "backup_id": "BK-20261012-163000-A8F2",
  "database": "production_db",
  "created_at": "2026-10-12T16:30:00+07:00",
  "source_agent": "c960acf5-08ca-4bf5-a175-92cc87e6d9f1",
  "server_version": "10.4.32-MariaDB",
  "dump_flags": ["--single-transaction", "--routines", "--triggers", "--
events", "--hex-blob", "--no-create-db"],
  "size": 325483129,
  "sha256": "a24b…",
  "tags": ["frequent", "hourly"]
}
```

`GET /status` response

```
{
  "agent_id": "…", "role": "node", "version": "0.1.0", "uptime_s": 86400,
  "store": {"path": "D:\\SevjalaBackups", "free_bytes": 512000000000},
  "health": "healthy",
  "backups": [{"backup_id": "BK-…", "size": 325483129, "sha256": "a24b…",
"state": "verified"}],
  "problems": []
}
```

Credentials: database password and pairing code live in the OS keyring (`zalando/go-keyring` : Credential Manager, Keychain, Secret Service); `password_ref` in config points at the entry. Linux without a keyring falls back to an AES-GCM file keyed from a root-only key file, logged as a warning.

Security model

LAN-only is not trusted-by-default: every LAN call is mutual TLS between explicitly paired peers, every local call goes through an OS-protected pipe, and nothing destructive runs without a typed confirmation.

Surface	Control
Discovery (UDP)	Read-only metadata; nonce-matched replies; no credentials, no store paths
Pairing	One-time 6-digit code, 10-minute TTL, 5 attempts, HMAC proof bound to both fingerprints; fingerprint seen in discovery must match the TLS cert
LAN API	mTLS, TLS 1.2+, both sides pin peer fingerprints; unknown cert = handshake failure before any HTTP; a node accepts one controller
Local API	Named pipe ACL (Administrators + SYSTEM) on Windows; socket mode 0660 + group on Unix; no network exposure
UI proxy	Binds 127.0.0.1 on a random port; one-time token in the launch URL exchanged for a session cookie; rejects requests whose `Host` is not `127.0.0.1:<port>`; CORS disabled
Secrets	DB password and pairing secrets in the OS keyring; never in config, logs, or `/status`
Backup files	SHA-256 verified on receipt and before restore; optional AES-256-GCM file encryption is Phase 3
Deletion	Nodes delete only on an authenticated `DELETE` with a reason, and log it; retention has `--dry-run`
Restore	New database by default; replacing production requires `--replace-production` plus typing the database name, and a pre-restore dump is taken first

Surface	Control
Logs	Rotating, persistent, record every event in PRD section 25; never contain secrets

Threats explicitly out of scope for v1: a compromised Controller host (it holds the DB credentials by design), and an attacker with admin rights on a Node (they can read the backup files; file encryption in Phase 3 addresses this).

Toolchain and build matrix

One source tree, modern Go, two toolchains: stock Go for every current OS and the XTLS/go-win7 patched SDK for the Windows 7 assets.

Asset	GOOS/GOARCH	Toolchain	Notes
`sevjala-windows-amd64.exe`	windows/amd64	stock Go	Windows 10/11
`sevjala-win7-amd64.exe`	windows/amd64	go-win7	Windows 7 64-bit, needs KB3125574
`sevjala-win7-386.exe`	windows/386	go-win7	Windows 7 32-bit
`sevjala-darwin-arm64` / `-amd64`	darwin	stock Go	tray needs cgo; build on macOS runner
`sevjala-linux-amd64`	linux/amd64	stock Go	tray needs libayatana-appindicator; agent/CLI do not

Rules

- `go.mod`: `go 1.27` (or whatever stock version is current at project start); everything must also compile with the go-win7 SDK of the same minor, so avoid packages that bind newer Windows APIs directly.
- Pure Go only in the agent and CLI: `modernc.org/sqlite`, `kardianos/service`, `Microsoft/go-winio`, `zalando/go-keyring`, `fyne.io/systray`. cgo is allowed only in the `tray` build tag on macOS/Linux.
- `CGO_ENABLED=0` for all Windows assets.

- The go-win7 SDK zip is downloaded by URL pinned to a release tag and verified against a SHA-256 recorded in the repo; CI fails on mismatch.
- `-race` tests run on Linux/macOS only (unsupported in the Win7 patch).
- Windows service build must include a manifest for `requestedExecutionLevel=asInvoker` and a version resource (`goversioninfo` or `go-winres`).
- Frontend (Phase 2): Node LTS, Svelte 5, Vite; `build.target = ['chrome109', 'firefox115']` ; output embedded via `//go:embed ui/dist` .

CI (GitHub Actions): `test` (unit + integration with a MariaDB service container), `build` matrix above, `smoke-win7` manual job that installs the service on a Windows 7 VM and runs `sevjala doctor` . Releases via GoReleaser with the two Windows 7 assets produced by a post-hook using the patched SDK.

Repository layout and project setup

One Go module at the repo root, `cmd/sevjala` as the single entry point, `internal/` packages matching the component list, and `ui/` as a sibling Vite project embedded at build time.

```
sevjala/
  cmd/sevjala/main.go          # cobra root; subcommands in
cmd/sevjala/cmd_*.go
  internal/
    identity/                  # agent id, cert/key, fingerprints, peers.json
    discovery/                 # UDP probe + responder
    pairing/                   # code issue/verify, HMAC proof
    lanapi/                    # mTLS server, client, handlers (identity,
pair, status, backups)
    localapi/                  # pipe/socket listener + routes shared with
lanapi handlers
    dump/                      # mysql detection, flags, streaming dump
    transfer/                  # worker pool, PUT client, retry/backoff
    store/                     # sqlite schema, migrations, queries
(controller)
    inventory/                 # node directory scan, manifests, .part
cleanup
    scheduler/                 # cron-like ticks, tag computation
    retention/                 # expected/actual diff, delete plan, dry-run
    reconcile/                 # status polling, offline detection, backfill
```

```
    restore/
    svc/                        # kardianos wrapper, firewall rules, data dirs
    tray/                       # build tag `tray`
    ui/                         # 127.0.0.1 proxy, token, embed of ui/dist
    config/                     # toml load/validate, keyring refs
    logx/                       # rotating logger, event emitter
  ui/                           # Svelte 5 + Vite (Phase 2)
  docs/                         # this blueprint exported, ADRs, protocol.md
  scripts/                      # build.sh / build.ps1, go-win7 fetch +
checksum
  .github/workflows/
  Makefile  go.mod  .goreleaser.yaml  README.md
```

Bootstrap

```
mkdir sevjala && cd sevjala && git init
go mod init github.com/<you>/sevjala
go get github.com/spf13/cobra github.com/kardianos/service \
       github.com/Microsoft/go-winio modernc.org/sqlite \
       github.com/zalando/go-keyring fyne.io/systray \
       github.com/BurntSushi/toml
mkdir -p cmd/sevjala
internal/{identity,discovery,pairing,lanapi,localapi,dump,transfer,store,inve
ntory,scheduler,retention,reconcile,restore,svc,tray,ui,config,logx} ui docs
scripts .github/workflows
```

Makefile targets: `build` (host), `build-all` (matrix), `build-win7` (patched SDK, checks checksum first), `test`, `test-integration` (docker MariaDB), `lint` (`golangci-lint`), `ui` (`npm ci && npm run build` in `ui/`), `release` (GoReleaser).

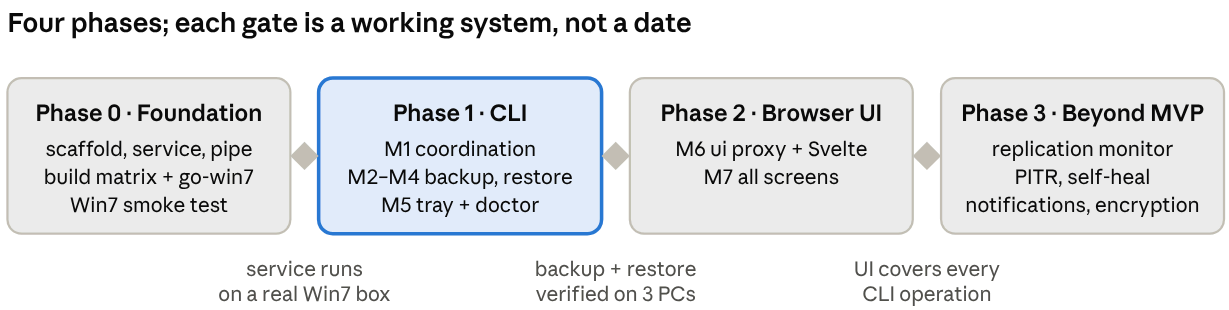
CLI surface (Phase 1)

```
sevjala agent [--role controller|node]      run in foreground (used by the
service)
sevjala service install|uninstall|start|stop|status
sevjala doctor                              ports, firewall, mysqldump,
disk, certs, pipe
sevjala discover                            list agents answering on the LAN
sevjala code                                (node) print a pairing code
sevjala pair <agent_id> --code N            (controller) pair a node
sevjala agents [add <host:port>|remove <id>]
sevjala ping <agent_id>
sevjala db detect|set
sevjala policy add|list|remove
sevjala backup now [--db X] [--tag T]
sevjala backups list [--matrix]
sevjala retention run [--dry-run]
sevjala restore <backup_id> --into <db> | --replace-production
sevjala status                              one-screen summary
sevjala logs [-f]
sevjala tray / sevjala ui                   Phase 1 M5 / Phase 2
```

Conventions: `gofmt` + `golangci-lint` clean; every package has table-driven tests; integration tests spin up two agents in-process on loopback with a MariaDB container; errors wrap with `%w`; logs are structured (`log/slog`) with `event` fields matching the PRD log list; no global state beyond the service entry point.

Roadmap

Phases are sequential and gated by a working system; milestones inside a phase are built in order because each one's API is what the next one uses.



Sevjala roadmap · 4 phases, 3 gates

Phase 1 is the product; Phases 2 and 3 add convenience on top of a stable pipe API and on-disk format.

Milestone	Scope	Exit criterion
P0 Foundation	Module, cobra CLI, `svc` wrapper, data dirs, rotating logs, empty pipe server, build matrix incl. go-win7, GoReleaser	`sevjala service install && sevjala service status` works on Win7, Win11, macOS, Linux
M1 Coordination	identity, discovery, pairing, mTLS LAN API, `/identity` `/pair` `/status`, `discover` `code` `pair` `agents` `ping`	`discover → pair → ping` across two real PCs; unpaired peer rejected at TLS
M2 First backup	db detect, dump pipeline, staging, transfer workers, node receive/verify, inventory scan, `backup now`, `backups list`	A 300 MB XAMPP DB lands `verified` on 3 nodes; a transfer killed mid-way leaves no `.part` after cleanup
M3 Unattended	scheduler, policies + tags, central retention + `--dry-run`, `/status` polling, offline/backfill, disk-full and checksum-mismatch handling	48 h run with one node powered off for 2 h: matrix correct, missing copies backfilled, retention counts exact
M4 Restore	`restore --into`, re-verify, `--replace-production` with pre-restore dump and typed confirmation	Restored DB `CHECKSUM TABLE` equals the original for every table
M5 Operator comfort	`fyne.io/systray` tray with state colour + menu, `doctor`, `status` one-screen	A non-engineer can tell protection is healthy from the tray alone
M6 UI shell	`sevjala ui` proxy, token + Host guard, SSE, Svelte 5 scaffold, Dashboard + Agents screens, tray "Open dashboard"	Dashboard matches `sevjala status` live
M7 UI screens	Backups matrix, Restore picker, Logs, Settings (policies, pairing approvals)	Every CLI operation is doable from the browser

Milestone	Scope	Exit criterion
P3 Beyond MVP	Replication monitor + screen, PITR via binlogs, self-heal between nodes, notifications, resumable transfer, file encryption, PostgreSQL	Each shipped as its own ADR + milestone

Agent handoff

Work one milestone per session, in order, against this blueprint; a coding agent that reads this section and the sections it points to has everything it needs to start P0 today.

How to use this document with a coding agent

1. Export this doc to Markdown as `docs/BLUEPRINT.md` and commit it; point the agent's project instructions (`CLAUDE.md` or equivalent) at it.
2. Open each session with: the milestone name, its row in the Roadmap table, and the sections it depends on (below). Nothing outside that scope is in play.
3. The agent must not change an Architecture decision (AD-xx) silently. A needed change is proposed as a new AD row with a reason, and work stops until it is accepted.
4. Anything the blueprint leaves unspecified is decided by the agent, recorded in `docs/decisions.md` as a one-liner, and raised in the session summary.

Milestone → sections

Milestone	Read first
P0	Components, Toolchain and build matrix, Repository layout
M1	Coordination protocol, Security model, Data model (identity, config, agents table)
M2	Backup engine (detection, dump, transfer), Data model (manifest, node store, backups tables)
M3	Backup engine (policies, retention, reconciliation), Coordination (`/status` polling)

Milestone	Read first
M4	Backup engine (restore), Security model (restore row)
M5	Components (tray), Coordination (local API)
M6–M7	Components (UI proxy), Security (UI proxy row), Toolchain (frontend rules)

Definition of done, every milestone

- [ ] Exit criterion from the Roadmap table demonstrated, with the command transcript pasted in the PR
- [ ] Unit tests for every new package; integration test on loopback for anything touching the LAN or local API
- [ ] `make lint test build-all` green, including the go-win7 assets
- [ ] `sevjala doctor` extended with any new check the milestone introduces
- [ ] Every event from PRD section 25 that the milestone can emit is logged with its `event` field
- [ ] `docs/protocol.md` updated when a wire message or on-disk format changed
- [ ] Manual smoke on a real Windows 7 machine for P0, M1, M2 and M5

Non-negotiables for the implementing agent

- Nodes never initiate LAN connections; Nodes never delete without an authenticated `DELETE`.
- No backup is reported `verified` before hash match and atomic rename.
- Secrets never appear in config files, logs, `/status`, or test fixtures.
- Pure Go and `CGO_ENABLED=0` for every Windows asset.
- Keep the CLI usable without the tray or UI; they are clients, not requirements.

Open questions to settle during P0/M1 (answer in `docs/decisions.md`)

- [ ] Exact MariaDB/MySQL versions bundled with the XAMPP releases in use, to pin `mysqldump` flag compatibility
- [ ] Whether SERVER-01 has KB3125574, which decides go-win7 1.27 vs 1.26.x for the Win7 assets
- [ ] Staging `keep` default (12) vs available disk on the Controller
- [ ] Whether a node may serve two controllers in v2 (affects `peers.json` shape now)